

Linear and Quadratic Programming

Contents

1. Overview	2
2. Linear programming	3
2.1 The problem	3
2.2 The answer is at a corner	3
2.3 The dual of a linear program	6
2.4 Reading the multipliers	7
3. Quadratic programming	8
3.1 The problem	8
3.2 Solving it with the KKT conditions	10
3.3 The dual of a quadratic program	11
3.4 Quadratic programs in ML	12
4. Summary	14
5. Sources	14
6. Key terms	14

Where this fits: Pipeline map step 0 (Foundations). See the [Course map](#).



- **Builds on:** Convex sets and convex optimisation ([Note MA-067](#)).

1. Overview

Key point: A linear program minimises a linear function over a region cut out by linear inequalities, and its answer sits at a corner; a quadratic program minimises a convex bowl over the same kind of region. Both are convex problems with a dual that is easy to write down.

This Note follows Chapter 7 (Sections 7.3.1 and 7.3.2) of *Mathematics for Machine Learning* (Deisenroth, Faisal and Ong, 2020).

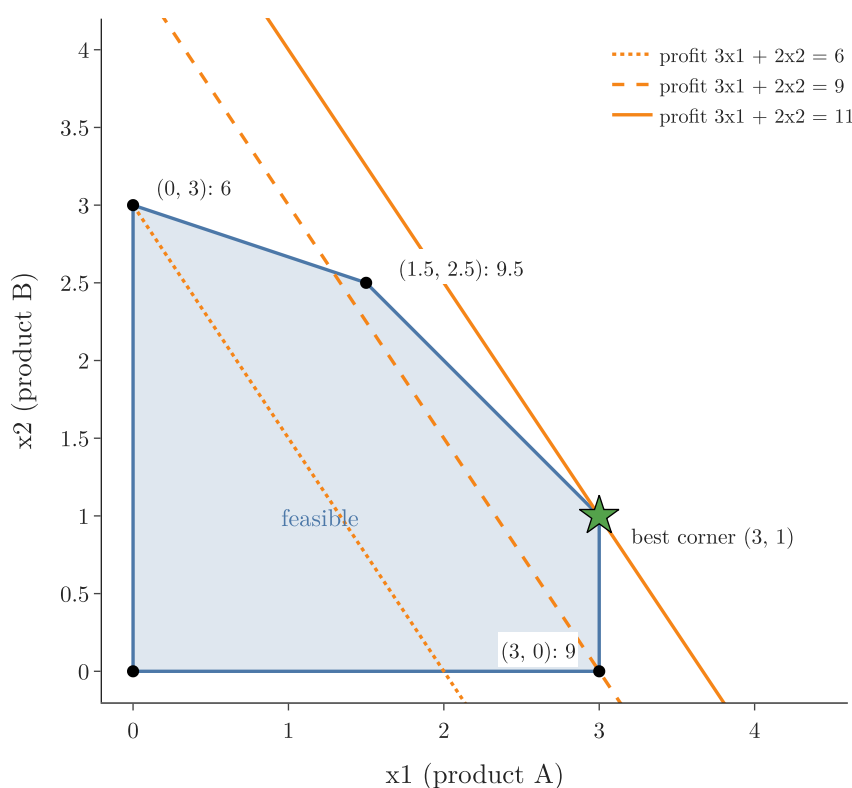


Figure 1: A linear program: the feasible region (blue) is a polygon, and the profit lines $3x_1 + 2x_2 = 6, 9, 11$ slide outward until the last one touches the region at a corner, $(3, 1)$

The [convex sets and functions Note](#) defined a convex optimisation problem. Two families of them are so common that they have their own names and their own solvers: **linear programs**

(G-1093) and **quadratic programs** (G-1598). Figure 1 shows a linear program.

For each family we write the problem, solve a small example from the picture, derive its dual with the Lagrangian of the [Lagrange multipliers Note](#), and check everything in Python.

2. Linear programming

Key point: Minimise $\mathbf{c}^\top \mathbf{x}$ subject to $A\mathbf{x} \leq \mathbf{b}$: a linear objective, linear inequality constraints, and an answer at a corner of the feasible polygon.

2.1 The problem

Key point: Everything is linear: the objective is a dot product with a fixed vector, and each constraint says a dot product is at most a number.

1. **In words:** choose d numbers to make a weighted sum as small as possible, while m other weighted sums stay below given limits.
2. **Formula:** a **linear program** is

$$\min_{\mathbf{x} \in \mathbb{R}^d} \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad A\mathbf{x} \leq \mathbf{b}$$

with A of size $m \times d$ and $\mathbf{b}$ of length m ; $A\mathbf{x} \leq \mathbf{b}$ means every row holds.

3. **Example:** a small workshop makes two products. Product A earns a profit of 3 thousand rupees per batch and product B earns 2. Each batch of either product needs one hour of oven time, and only 4 hours are free. A batch of A needs 1 bag of flour and a batch of B needs 3, with 9 bags in stock. At most 3 batches of A can be sold. To maximise profit $3x_1 + 2x_2$, we minimise its negative:

$$\min -3x_1 - 2x_2 \quad \text{subject to} \quad x_1 + x_2 \leq 4, \quad x_1 + 3x_2 \leq 9, \quad x_1 \leq 3, \quad -x_1 \leq 0, \quad -x_2 \leq 0$$

So $\mathbf{c} = [-3, -2]^\top$, $\mathbf{b} = [4, 9, 3, 0, 0]^\top$, and A has the rows $[1, 1]$, $[1, 3]$, $[1, 0]$, $[-1, 0]$, $[0, -1]$.

Each constraint is a half-plane, and the feasible region is their overlap: a convex polygon (in more dimensions, a **polytope**, G-1518). The objective is linear, so it is convex too, and a linear program is a **convex optimisation problem** (G-478).

2.2 The answer is at a corner

Key point: The contour lines of a linear objective are parallel straight lines; sliding them in the improving direction, the last feasible point they touch is a corner of the polygon.

Figure 1 is the level-curve picture of the [Lagrange multipliers Note](#) with straight lines instead of ellipses. In Figure 1, the lines of equal profit $3x_1 + 2x_2 = p$ are parallel. Raising p slides them up and to the right. The line $p = 11$ is the last one that still touches the region, at the

single corner $(3, 1)$. Think of pushing a ruler across a cut-out cardboard shape while keeping it parallel to itself: the last bit of cardboard under the ruler is a corner (or a whole edge).

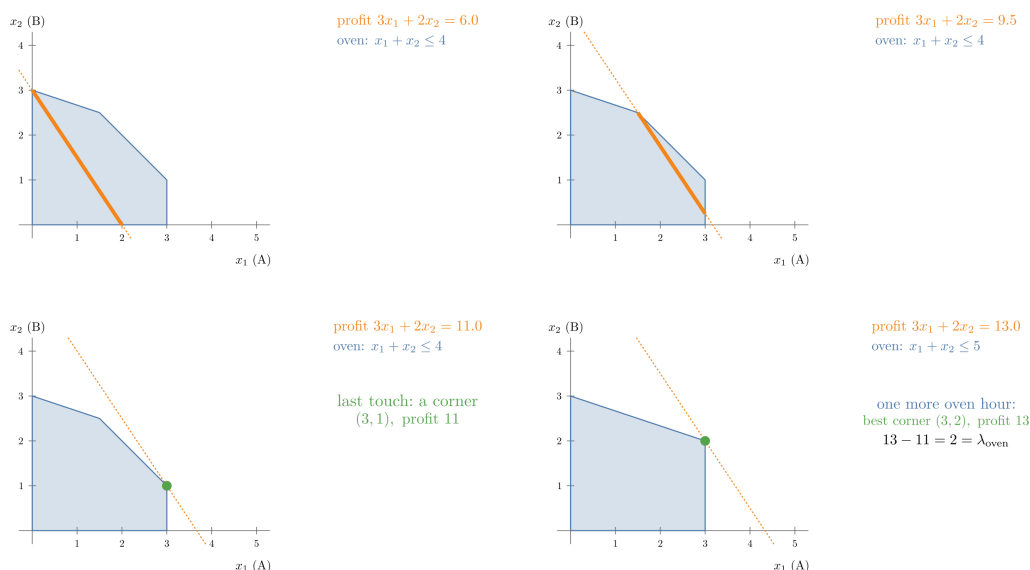


Figure 2: The profit line $3x_1 + 2x_2 = p$ slides outward; its feasible part (thick orange) shrinks until it touches only the corner $(3, 1)$ at $p = 11$. Then one more oven hour moves the best corner to $(3, 2)$, profit 13. Key frames: $p = 6, 9.5, 11$, and oven limit 5

Figure 2 runs the ruler. Watch the thick orange part of the line: it is the set of plans that earn exactly p , and it shrinks to a single point at the last corner. The last frame previews Section 2.4.

A second way to see it needs no ruler. A corner of the region is called a **vertex** (G-2258).

1. **Along an edge** the profit changes in a straight line. On the edge from $(3, 0)$ to $(3, 1)$ it rises evenly from 9 to 11. A point between two vertices always earns something in between, so it is never better than the better end.
2. **Inside the region** there is still room to make a little more of A or of B, and each extra batch adds profit. So an inside point is never the best.

Only the vertices are left. Because the answer is at a corner, checking the corners is enough for a small problem:

Corner	Profit $3x_1 + 2x_2$
$(0, 0)$	0
$(3, 0)$	9
$(3, 1)$	11
$(1.5, 2.5)$	9.5
$(0, 3)$	6

The best plan is 3 batches of A and 1 of B, for 11 thousand rupees. If the profit lines were parallel to an edge, every point of that edge would tie, but a corner would still be among the best points. With many products and many constraints there are far too many vertices to list. The **simplex algorithm** (G-2259) avoids listing them: it walks from one vertex to a neighbouring vertex, and only ever to a better one. Figure 3 first cuts out the region one constraint at a time and then runs the walk on the workshop.

1. Start at $(0,0)$, profit 0.
2. Choose a direction. A batch of A earns 3 and a batch of B earns 2, so move along x_1 until a constraint stops us: the vertex $(3,0)$, profit 9.
3. Look at the next neighbour, $(3,1)$: profit 11, better. Move there.
4. Look at the next neighbour, $(1.5, 2.5)$: profit 9.5, worse. No neighbour is better, so stop.

The walk visited 3 of the 5 vertices and ends at the same answer as the table, $(3,1)$ with profit 11. Stopping is safe because a linear program is a convex problem, and in a convex problem every local minimum is a global minimum (see the [convex sets and functions Note](#)): a vertex with no better neighbour is the best of all.

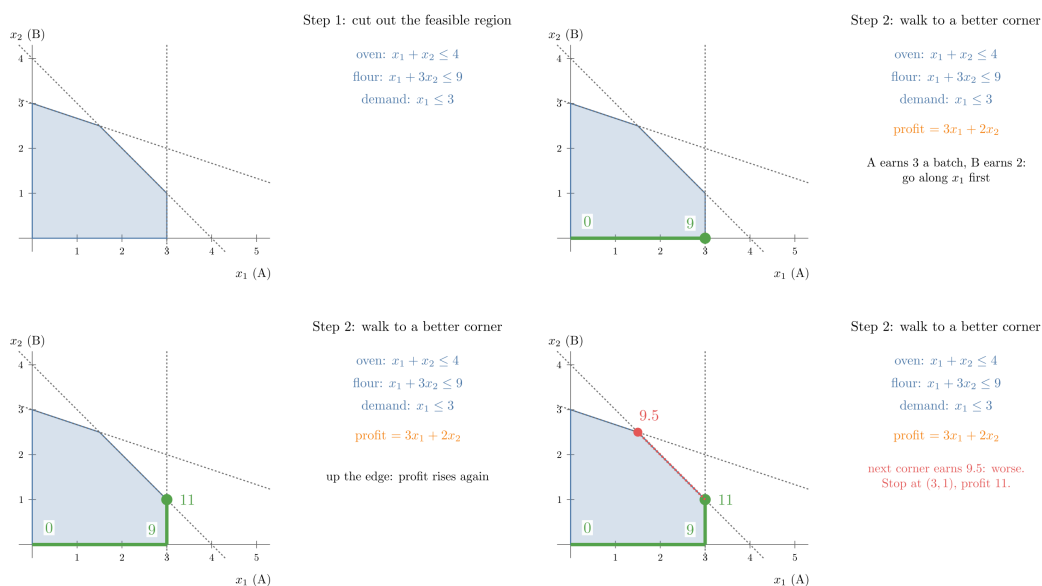


Figure 3: The workshop region cut out one constraint at a time (oven, flour, demand), then the walk from vertex to vertex: profit 0, 9, 11. The next vertex earns 9.5 (red), so the walk stops at $(3,1)$. Idea after StatQuest, "Optimization with Linear Programming (and the Simplex Algorithm), Main Ideas!!!"; our own problem

Real solvers use the simplex algorithm or interior-point methods (for the latter, Boyd and Vandenberghe, Ch. 11).

2.3 The dual of a linear program

Key point: The dual of $\min \mathbf{c}^\top \mathbf{x}$ subject to $A\mathbf{x} \leq \mathbf{b}$ is $\max -\mathbf{b}^\top \boldsymbol{\lambda}$ subject to $\mathbf{c} + A^\top \boldsymbol{\lambda} = \mathbf{0}$ and $\boldsymbol{\lambda} \geq \mathbf{0}$: another linear program, with one variable per constraint.

We follow the recipe of the [Lagrange multipliers Note](#) (Section 6): build the Lagrangian, minimise over $\mathbf{x}$, and maximise the result over $\boldsymbol{\lambda} \geq \mathbf{0}$.

1. **In words:** the Lagrangian is linear in $\mathbf{x}$. A linear function of $\mathbf{x}$ has a finite minimum only if its slope is zero, so the dual keeps only the multipliers that make the slope vanish.
2. **Formula:** the Lagrangian, with the $\mathbf{x}$ terms gathered, is

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (A\mathbf{x} - \mathbf{b}) = (\mathbf{c} + A^\top \boldsymbol{\lambda})^\top \mathbf{x} - \boldsymbol{\lambda}^\top \mathbf{b}$$

Its gradient in $\mathbf{x}$ is $\mathbf{c} + A^\top \boldsymbol{\lambda}$. If that is not zero, $\mathcal{L}$ falls without limit; if it is zero, $D(\boldsymbol{\lambda}) = -\mathbf{b}^\top \boldsymbol{\lambda}$. The dual problem is

$$\max_{\boldsymbol{\lambda} \in \mathbb{R}^m} -\mathbf{b}^\top \boldsymbol{\lambda} \quad \text{subject to} \quad \mathbf{c} + A^\top \boldsymbol{\lambda} = \mathbf{0}, \quad \boldsymbol{\lambda} \geq \mathbf{0}$$

3. **Example:** for the workshop, $\boldsymbol{\lambda} = [2, 0, 1, 0, 0]^\top$ (one value per constraint, in the order oven, flour, demand, $x_1 \geq 0$, $x_2 \geq 0$). Check the equality:

$$\mathbf{c} + A^\top \boldsymbol{\lambda} = \begin{bmatrix} -3 \\ -2 \end{bmatrix} + 2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The dual value is $-\mathbf{b}^\top \boldsymbol{\lambda} = -(4 \times 2 + 3 \times 1) = -11$, the same as the primal minimum $\mathbf{c}^\top \mathbf{x} = -11$. Strong duality holds, as it does for every linear program whose primal problem is feasible (Boyd and Vandenberghe, Ch. 5).

Figure 4 shows what the equality $\mathbf{c} + A^\top \boldsymbol{\lambda} = \mathbf{0}$ means at the best corner. Each active constraint has a normal arrow pointing out of the region: $[1, 1]$ for the oven and $[1, 0]$ for demand. The profit direction $[3, 2]$ is 2 oven arrows plus 1 demand arrow. Every way of raising profit therefore pushes against an active wall, which is why no feasible move can improve on the corner; the weights 2 and 1 are the multipliers.

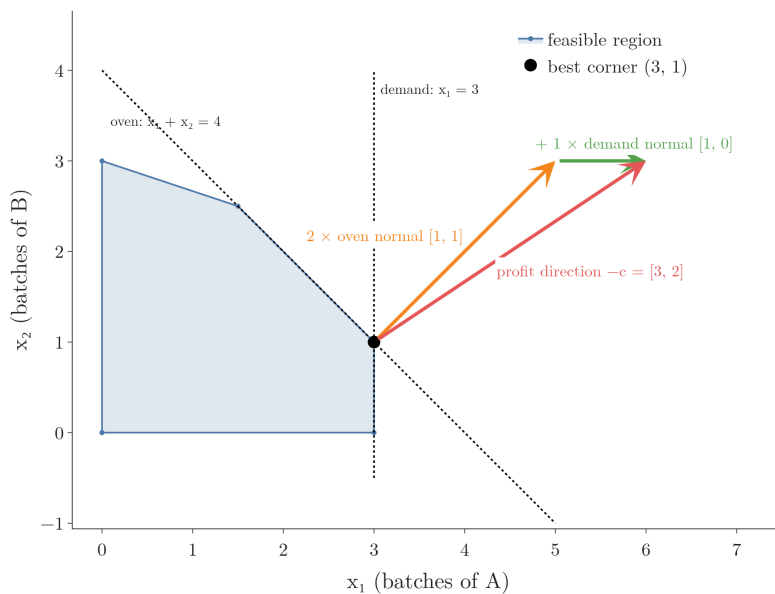


Figure 4: The dual condition at the best corner $(3, 1)$. The profit direction $-c = [3, 2]$ (red) equals 2 times the oven normal $[1, 1]$ (orange) plus 1 times the demand normal $[1, 0]$ (green): non-negative weights, the multipliers 2 and 1.

The primal has d variables and m constraints; the dual has m variables and d equality constraints. We can solve whichever is smaller. By convention the primal is minimised and the dual maximised.

2.4 Reading the multipliers

Key point: Each multiplier is the extra profit one more unit of that resource would bring; a resource with spare capacity is worth 0.

The multipliers are the shadow prices of the [Lagrange multipliers Note](#) (Section 4.2). The last frame of Figure 2 shows the oven's multiplier: one more oven hour moves the best corner from $(3, 1)$ to $(3, 2)$ and the profit from 11 to 13. Changing one limit by one unit and solving again confirms each one:

Constraint	At the answer	Multiplier	Re-solved with one more unit
Oven: $x_1 + x_2 \leq 4$	$4 = 4$, active	2	profit 13 (up by 2)
Flour: $x_1 + 3x_2 \leq 9$	$6 < 9$, inactive	0	profit 11 (no change)
Demand: $x_1 \leq 3$	$3 = 3$, active	1	profit 12 (up by 1)

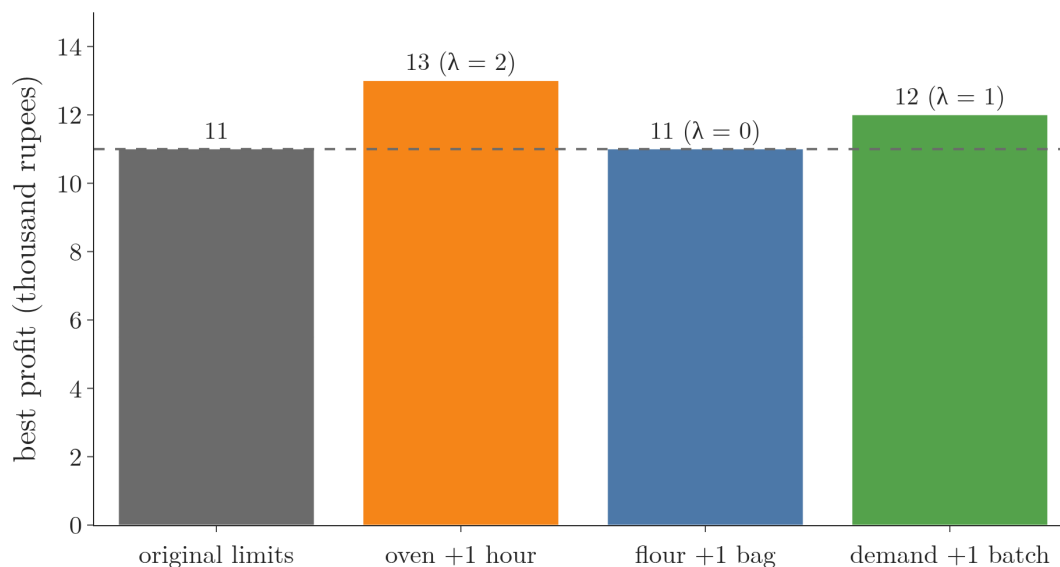


Figure 5: The best profit re-solved with one more unit of each resource. Oven: 13, up by 2. Flour: 11, unchanged. Demand: 12, up by 1. The rises are the multipliers.

Figure 5 draws the last column of the table. Flour is not the bottleneck: 3 bags are left over, so more flour is worth nothing. An extra oven hour is worth 2 thousand rupees, so the workshop should pay up to that much for one. The zero multiplier on flour is complementary slackness in action: the inactive constraint has multiplier 0.

Extra: Linear programs appear in ML too. Fitting a line by minimising the sum of absolute errors $\sum_i |y_i - \mathbf{w}^\top \mathbf{x}_i|$ (least absolute deviations) becomes a linear program by giving each **observation** (G-1374; one record, one row of the data table) an extra variable $t_i \geq |y_i - \mathbf{w}^\top \mathbf{x}_i|$, written as two linear inequalities, and minimising $\sum_i t_i$. scikit-learn’s `QuantileRegressor` solves this kind of linear program with `scipy.optimize.linprog` (scikit-learn docs, `QuantileRegressor`).

3. Quadratic programming

Key point: Minimise $\frac{1}{2} \mathbf{x}^\top Q \mathbf{x} + \mathbf{c}^\top \mathbf{x}$ subject to $A \mathbf{x} \leq \mathbf{b}$, with Q positive definite: a convex bowl over a polygon, whose answer can lie inside the region, on an edge, or at a corner.

3.1 The problem

Key point: Same linear constraints as a linear program, but the objective is a bowl, so its level curves are ellipses.

1. **In words:** minimise a quadratic bowl over a region cut out by linear inequalities.

2. **Formula:** a quadratic program is

$$\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2} \mathbf{x}^\top Q \mathbf{x} + \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad A \mathbf{x} \leq \mathbf{b}$$

with Q symmetric and **positive definite** (G-1530; all eigenvalues positive), so the objective is a strictly convex bowl (see the [convex sets and functions Note](#), Section 4.3). The term $\frac{1}{2} \mathbf{x}^\top Q \mathbf{x}$ is a **quadratic form** (G-1597) (see the [linear algebra roadmap Note](#)).

3. **Example:** $Q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ (eigenvalues 1 and 3) and $\mathbf{c} = [-8, -7]^\top$, so the objective is $x_1^2 + x_1x_2 + x_2^2 - 8x_1 - 7x_2$. The constraints are $x_1 + x_2 \leq 2$, $x_1 \geq 0$ and $x_2 \geq 0$: a triangle.

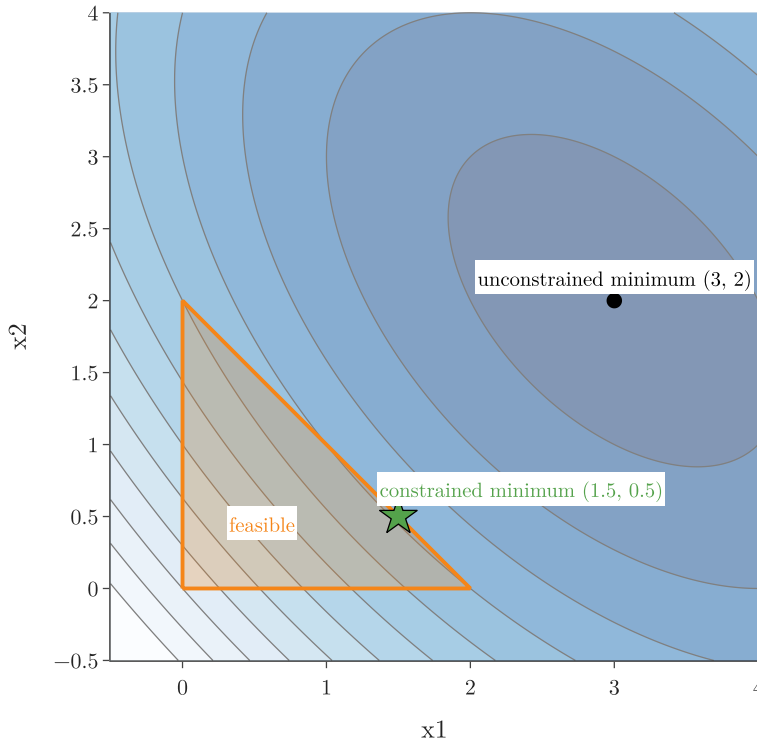


Figure 6: A quadratic program: the elliptical contours of a bowl with its unconstrained minimum at $(3, 2)$, and the triangular feasible region (orange). The constrained minimum (star) lies on the edge $x_1 + x_2 = 2$

Without constraints, the gradient $Q\mathbf{x} + \mathbf{c}$ is zero at $\mathbf{x} = -Q^{-1}\mathbf{c} = (3, 2)$. That point is outside the triangle, since $3 + 2 = 5 > 2$. So the answer lies on the boundary: on the edge $x_1 + x_2 = 2$, where a contour ellipse just touches it (Figure 6). It does not have to be a corner, as it would for a linear program.

3.2 Solving it with the KKT conditions

Key point: Guess which constraints are active, solve the resulting equations, then check that every multiplier is non-negative.

Only the edge constraint $x_1 + x_2 \leq 2$ is active, with multiplier λ . The stationarity condition $Q\mathbf{x} + \mathbf{c} + \lambda[1, 1]^T = \mathbf{0}$ and the active constraint give three equations:

$$2x_1 + x_2 - 8 + \lambda = 0, \quad x_1 + 2x_2 - 7 + \lambda = 0, \quad x_1 + x_2 = 2$$

Subtracting the second from the first gives $x_1 - x_2 = 1$; with $x_1 + x_2 = 2$ this means $x_1 = 1.5$, $x_2 = 0.5$. Then $\lambda = 8 - 3 - 0.5 = 4.5$.

Figure 7 shows the stationarity condition at the answer. The downhill direction $-\nabla f = [4.5, 4.5]$ points straight out of the edge, exactly $\lambda = 4.5$ times the edge's normal $[1, 1]$. It has no part along the edge, so sliding along the edge cannot lower f , and stepping out of the triangle is not allowed.

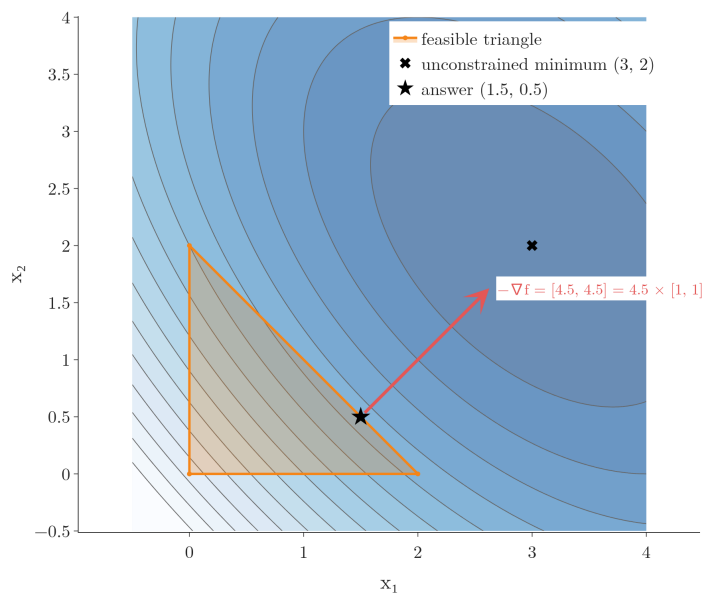


Figure 7: The quadratic program at its answer $(1.5, 0.5)$ (star). The downhill direction $-\nabla f = [4.5, 4.5]$ (red) is 4.5 times the normal of the active edge $x_1 + x_2 = 2$; the unconstrained minimum $(3, 2)$ lies outside the triangle.

The checks of the **KKT conditions** (G-1013): both coordinates are positive, so the two sign constraints are inactive with multiplier 0; $\lambda = 4.5 \geq 0$. The objective value is $2.25 + 0.75 + 0.25 - 12 - 3.5 = -12.25$.

3.3 The dual of a quadratic program

Key point: Minimising the Lagrangian over $\mathbf{x}$ gives $\mathbf{x} = -Q^{-1}(\mathbf{c} + A^T\boldsymbol{\lambda})$, and putting it back leaves a concave quadratic in $\boldsymbol{\lambda}$ with only the constraints $\boldsymbol{\lambda} \geq \mathbf{0}$.

1. **In words:** the Lagrangian is a bowl in $\mathbf{x}$; its lowest point has a formula, and substituting that formula leaves a function of the multipliers alone.
2. **Formula:** the Lagrangian is

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2}\mathbf{x}^T Q \mathbf{x} + (\mathbf{c} + A^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b}$$

Setting its gradient $Q\mathbf{x} + \mathbf{c} + A^T\boldsymbol{\lambda}$ to zero gives $\mathbf{x} = -Q^{-1}(\mathbf{c} + A^T\boldsymbol{\lambda})$; Q is invertible because it is positive definite. Substituting:

$$D(\boldsymbol{\lambda}) = -\frac{1}{2}(\mathbf{c} + A^T \boldsymbol{\lambda})^T Q^{-1}(\mathbf{c} + A^T \boldsymbol{\lambda}) - \boldsymbol{\lambda}^T \mathbf{b}, \quad \text{dual: } \max_{\boldsymbol{\lambda} \geq \mathbf{0}} D(\boldsymbol{\lambda})$$

3. **Example:** at $\boldsymbol{\lambda} = [4.5, 0, 0]^T$ (edge, $x_1 \geq 0, x_2 \geq 0$), $\mathbf{c} + A^T \boldsymbol{\lambda} = [-8 + 4.5, -7 + 4.5]^T = [-3.5, -2.5]^T$. With $Q^{-1} = \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$:

$$\mathbf{x} = -\frac{1}{3} \begin{bmatrix} 2(-3.5) - (-2.5) \\ -(-3.5) + 2(-2.5) \end{bmatrix} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}, \quad D = -\frac{1}{2} \times 6.5 - 4.5 \times 2 = -12.25$$

where $6.5 = \frac{1}{3}(2 \times 12.25 - 2 \times 8.75 + 2 \times 6.25)$. The dual maximum equals the primal minimum -12.25 .

Figure 8 draws D along the edge multiplier, with the two sign-constraint multipliers held at 0. It is an upside-down bowl that stays below the primal minimum -12.25 (weak duality) and reaches it exactly at $\lambda = 4.5$ (strong duality).

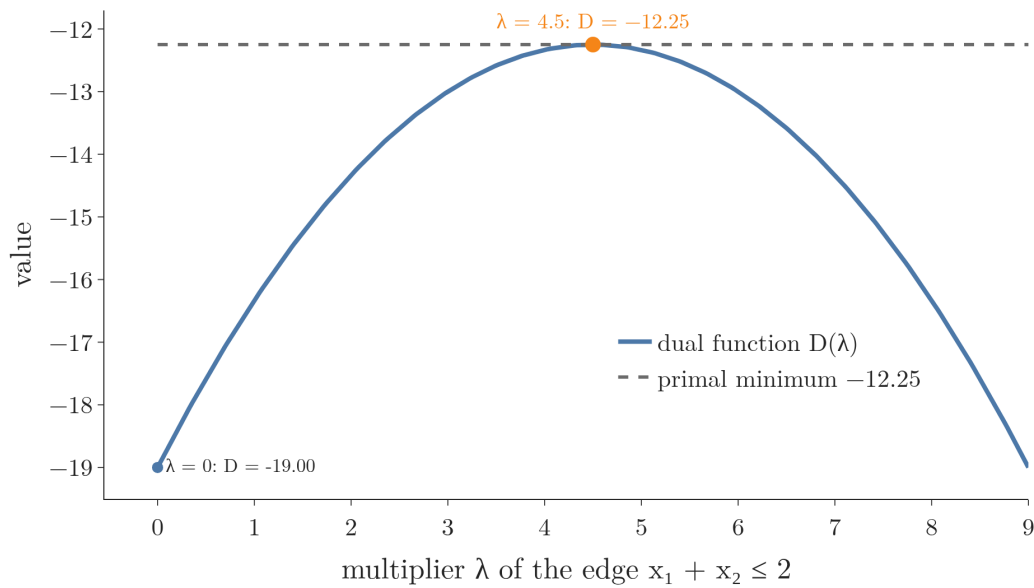


Figure 8: The dual function of the quadratic program along the edge multiplier λ (the other two multipliers at 0): a concave curve below the primal minimum -12.25 (dashed), touching it at $\lambda = 4.5$.

The dual has only simple sign constraints $\boldsymbol{\lambda} \geq \mathbf{0}$. They are easy to keep: after a gradient step, setting every negative multiplier to 0 restores them.

3.4 Quadratic programs in ML

Key point: Training a support vector machine is a quadratic program, in its primal form and in its dual form.

- **SVM, primal (support vector machine, G-1921):** minimise $\frac{1}{2}\|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ (see the [SVM soft margin Note](#)). The objective is quadratic and each constraint is linear in $(\mathbf{w}, b)$. Here Q is only positive semi-definite, because b has no squared term, so the dual is derived as in the [Lagrange multipliers Note](#) (Section 7.2) rather than with Q^{-1} .
- **SVM, dual:** maximise $\sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j$ subject to $\alpha_i \geq 0$ and $\sum_i \alpha_i y_i = 0$: a quadratic program in the multipliers. The soft-margin version only adds the upper bound $\alpha_i \leq C$. scikit-learn’s `SVC` solves this dual (scikit-learn User Guide, SVM).
- **Lasso, constraint form:** least squares subject to $\sum_i |w_i| \leq t$ is a quadratic program once each weight is split into a positive and a negative part (Tibshirani 1996, §6).

Python: scipy’s `linprog` solves linear programs and reports the multipliers in `ineqlin.marginals` (with scipy’s sign, so negative here). `minimize` with `trust-constr` handles the quadratic program.

```
import numpy as np
from scipy.optimize import linprog, minimize, LinearConstraint
```

```

# Linear program: the workshop
c = [-3, -2]
A = [[1, 1], [1, 3], [1, 0]]
b = [4, 9, 3]
lp = linprog(c, A_ub=A, b_ub=b, bounds=[(0, None)] * 2)
lp.x, lp.fun          # [3. 1.], -11.0
lp.ineqlin.marginals  # [-2. -0. -1.]: oven, flour, demand

# Quadratic program: the triangle
Q = np.array([[2, 1], [1, 2]])
q = np.array([-8, -7])
f = lambda x: 0.5 * x @ Q @ x + q @ x
edge = LinearConstraint([1, 1], -np.inf, 2)
qp = minimize(f, [0, 0], method="trust-constr",
              constraints=[edge], bounds=[(0, None)] * 2)
qp.x.round(3), round(qp.fun, 3)  # [1.5 0.5], -12.25

```

4. Summary

	Linear program	Quadratic program
Objective	$\mathbf{c}^\top \mathbf{x}$	$\frac{1}{2} \mathbf{x}^\top Q \mathbf{x} + \mathbf{c}^\top \mathbf{x}$, Q positive definite
Constraints	$A\mathbf{x} \leq \mathbf{b}$	$A\mathbf{x} \leq \mathbf{b}$
Contours	parallel straight lines	ellipses
Where the answer is	a corner of the polygon	inside, on an edge or at a corner
Dual	$\max -\mathbf{b}^\top \boldsymbol{\lambda}$, $\mathbf{c} + A^\top \boldsymbol{\lambda} = \mathbf{0}$, $\boldsymbol{\lambda} \geq \mathbf{0}$	$\max D(\boldsymbol{\lambda})$ (concave quadratic), $\boldsymbol{\lambda} \geq \mathbf{0}$
Example	workshop: (3, 1), profit 11	triangle: (1.5, 0.5), value -12.25
In ML	least absolute deviations, quantile regression	SVM (primal and dual), Lasso in constraint form

- Both are convex problems with linear constraints, so whenever the primal is feasible the dual value equals the primal value (Boyd and Vandenberghe, Ch. 5).
- Multipliers are shadow prices: an inactive constraint has multiplier 0.

5. Sources

Built from

- Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020, Chapter 7.
- StatQuest with Josh Starmer, “Optimization with Linear Programming (and the Simplex Algorithm), Main Ideas!!!”, YouTube, <https://www.youtube.com/watch?v=h5o1n1QMcmM> (Section 2.2, Figure 3)

Other references

- Boyd and Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Chapter 5 (duality; strong duality for linear constraints, Sec. 5.2.3 to 5.2.4), Chapter 11 (interior-point methods).
- Tibshirani, “Regression Shrinkage and Selection via the Lasso”, *Journal of the Royal Statistical Society B*, 1996, Section 6.
- scikit-learn documentation: `QuantileRegressor`; User Guide, Support Vector Machines, Mathematical formulation.

6. Key terms

Term	Meaning
Vertex (G-2258)	A corner of the feasible region; a linear program always has a best point at a vertex
Simplex algorithm (G-2259)	Solving a linear program by walking from vertex to neighbouring vertex, always to a better one, until no neighbour is better
Linear program	Minimising a linear function subject to linear inequality constraints
Polytope	The region where a set of linear inequalities all hold: a polygon in two dimensions
Quadratic program	Minimising a convex quadratic function subject to linear inequality constraints
Positive definite matrix	A symmetric matrix whose eigenvalues are all positive; its quadratic form is a strictly convex bowl